POX LEARNING SWITCH LOGIC

Handling Packet-In Events for Unknown Destination MAC Addresses

---

OVERVIEW

In POX, a basic learning switch application intercepts packet events sent from OpenFlow switches to build a dynamic MAC-to-port binding table (`mac_to_port` dictionary). Upon receiving an **ofp_packet_in** message containing an unknown destination MAC address, the controller executes a specific sequential logic to learn the source location and handle packet delivery.

SEQUENTIAL CONTROLLER EXECUTION LOGIC

1. **Parse Event Data:** The controller extracts the OpenFlow packet object and ingress port from the event (`event.parsed` and `event.port`). It parses header attributes, specifically extracting the Source MAC address (`eth.src`) and Destination MAC address (`eth.dst`).
2. **MAC Learning (Source Address Binding):** The controller updates its internal forwarding table mapping for that specific switch DPID (`mac_to_port[event.dpid][eth.src] = event.port`). This records the port on which the source host resides for future lookup.
3. **Destination Lookup:** The controller queries its `mac_to_port` table for the Destination MAC address (`eth.dst`). Since the destination is **unknown** (not present in the table), the lookup fails.
4. **Construct Flood Action:** Because the destination port is unknown, the controller creates an OpenFlow output action set to flood the packet across all ports except the receiving ingress port (`ofp_action_output(port = OFPP_FLOOD)`).
5. **Construct Packet-Out Message:** The controller instantiates an **ofp_packet_out** message structure, setting:
   - `msg.actions.append(ofp_action_output(port = OFPP_FLOOD))`
   - `msg.in_port = event.port` (to avoid sending back out the ingress interface)
   - `msg.data = event.ofp` or `msg.buffer_id = event.ofp.buffer_id` (attaching the raw packet payload or switch buffer reference)
6. **Send Message to Switch:** The controller calls `event.connection.send(msg)` to transmit the `ofp_packet_out` instruction down to the switch.

KEY OPERATIONAL OUTCOME

By executing this logic, the switch floods the frame so it reaches its intended destination host while the controller silently learns the location of the source host. Once the destination host eventually replies, another **ofp_packet_in** will trigger, allowing the controller to learn the second host's port and install a direct **ofp_flow_mod** entry to bypass future controller intervention.